(*)

GRAPHS

(.) tree is also a graph with no cycles

(.) Forest is a collection of trees.

(.) Density of a graph is the propotion of the possible pair of vertices connected by edges.

(-) A sparse graph has relatively few of the possible edges present

(-) A dense graph has relatively few of the possible edges missing.

(.) 3 ways to represent Graphs
-> Adjacency list
-> Adjacency Matrix
-> Array of Edges.

GRAPHS

(.) Vertices connected by an edge are adjacent vertices

(.) Degree of a vertex is the no# of ~~edges~~ adjacent vertices / edges connected to a vertex.

(.) Weights are the costs that go from one vertex to the other

(.) Path is a seq. of vertices

(.) Simple path has no repeated vertices

(.) Cycle is a ~~pe~~ simple path except that the first vertex & the last vertex are the same

(.) Acyclic graph is a graph with no cyles.

(.) Any two vertices that are connected by some path form a connected graph.

(.) A graph is connected if there is a path from every vertex to every other vertex in the graph.

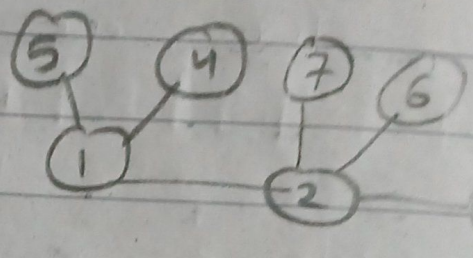
(.) A graph that is not connected consists of ~~minimal~~ a set of connected components ~~that~~ which are maximal connected subgraphs.

BFS & DFS

1) Visiting a Vertex :- going on a particular vertex
2) Exploration of Vertex :- If on a particular vertex then we visit all of it's adjacent vertices

Ex#01

1, 2, 4, 5, 3, 7, 6



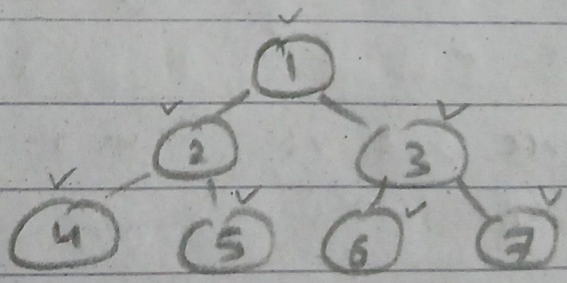
BFS :-

(+) Can select any as Starting vertex
(-) visit the source vertex
(-) Start exploring immediately (all adjacent vertices)
(-) exploration may be done in any order
(-) Once we visit a vertex mark it as visited bcz graph unlike tree and a hierarical structure, and so graphs may have cycles so to not visit any vertex twice.
(-) Similar to level order traversal of BST
(-) output seen is the same as traversing the graph level by level wrt the source node.

DFS :- 1, 2, 3, 6, 7, 4, 5

(.) can select any starting vertex
(.) visit source vertex.
(.) start exploration by going to A adjacent vertex only
(.) suspend exploration then & there, go deep in the branch.
(.) We've reached a new vertex so we start exploring that vertex, & suspend the exploration of ~~pre~~ adjacent vertices to the previous node.
(.) Immediately explore & visit the 1st vertex that appears
(.) Similar to pre order traversal in BST.

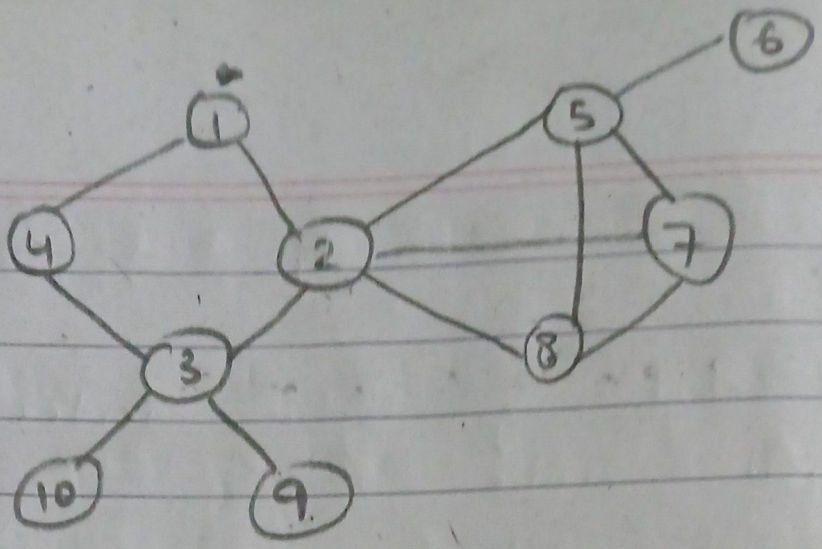
Ex # 02



BFS :- 1, 2, 3, 4, 5, 6, 7

DFS :- 1, 2, 4, 5, 3, 6, 7

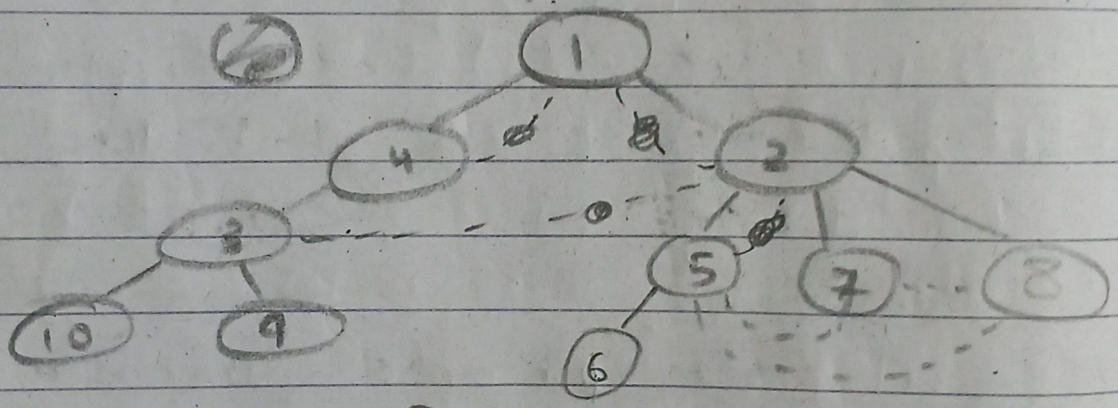
Ex#03



BFS

1, 4, 2, 3, 5, 7, 8, 10, 9, 6

Q | ~~1~~ | ~~4~~ | ~~2~~ | ~~3~~ | ~~5~~ | ~~7~~ | ~~8~~ | ~~10~~ | 9 | 6 |



BFS Spanning Tree

--- (dotted edges) are called cross edges.

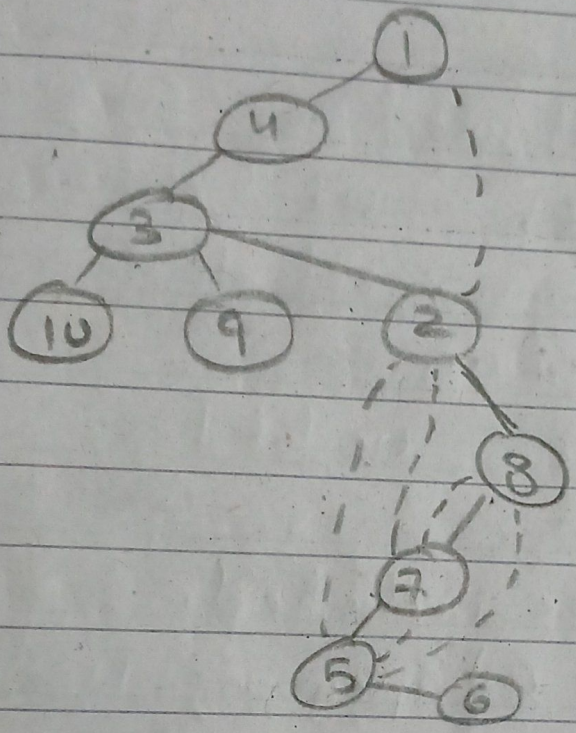
RULE :- When vertex selected for exploration we must visit all of the adjacent vertices then only we can go to next vertex for exploration.

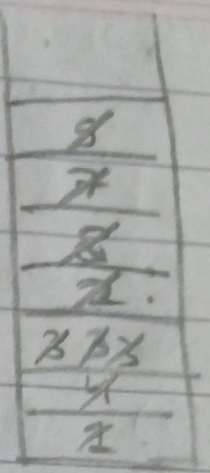
TIME COMPLEXITY = O(n)

DFS

this should be in reverse order

1, 4, 3, 10, 9, 2, 8, 7, 5, 6





Stack

→ DFS Spanning Tree

→ (---) dotted edges are called back edges.

RULE :- when we have visited a new vertex suspend current exploration & start exploring new vertex.

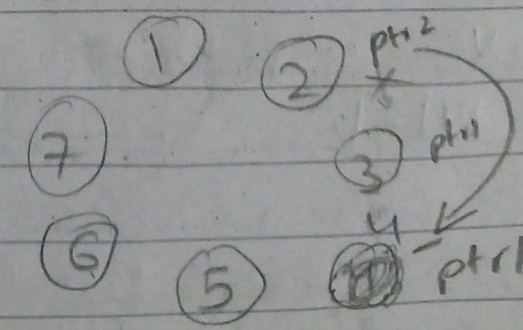
no # of nodes / vertices

TIME COMPLEXITY = O(n)

F'23 DS - Final

Q4) Josephus problem

via circular linked list.



MINIMUM SPANNING TREES (MST)

(.) A tree will not have a cycle.

(.) A Spanning tree is a subgraph of a graph.

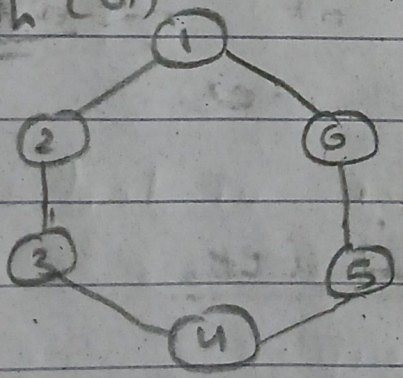
(.) Graph is represented as $G = (V, E)$

$V = \{1, 2, 3, 4, 5, 6\}$ → set of vertices

$E = \{(1,2), (2,3), \ldots\}$ → set of edges

(.) A Spanning tree would have the subset of edges & all vertices must be included.

graph (G)



$|V| = 6$

$|E| = 6$

$S \subseteq G$

$S = (V', E')$

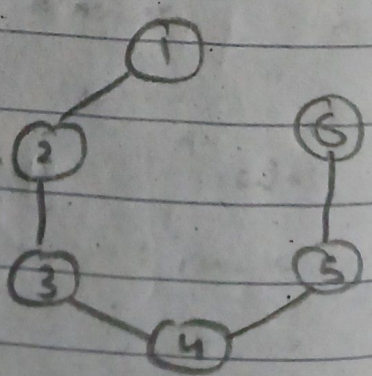
$V' = V$

$E' = |V| - 1$

(S) Spanning Tree is is a subgraph of a graph having (n) all vertices but only $n-1$ edges

$|V| = 6 = n$

$|E| = n-1 = 5$

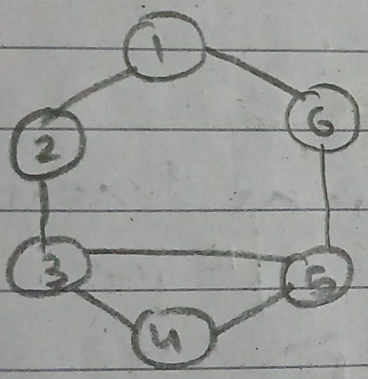


Q) For a given graph how many different spanning trees can be generated? (possibly)

A) our graph has n = ~~edges~~ vertices we must only have $(n-1)$ edges in the minimum cost spanning tree

${}^{n}C_{n-1}$ in prev example ${}^{6}C_{5} = 6$

$\Rightarrow {}^{|E|}C_{|V|-1}$ - not of cycles



$= {}^{7}C_{5} - 2$

↳ not of possible spanning trees.

WEIGHTED GRAPH

(*) A Graph where edges have weights

(•) The spanning trees now will have different costs as we have weighted graphs which can have numerous / varying spanning trees.

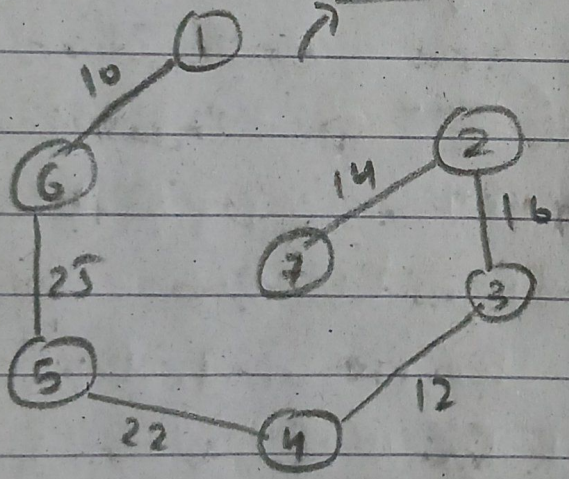
(∞) Greedy Methods for finding (MSCST)

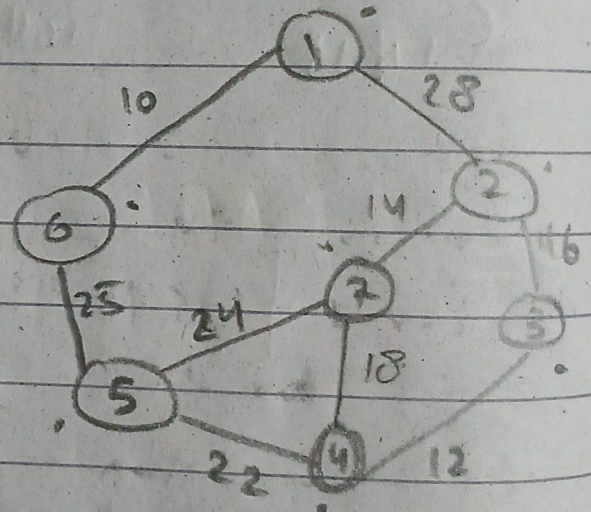
1 -> Prim's algo 2 -> kruskal's algo

PRIMS

1) First we select the minimum cost edge from Graph.
2) Now again select the minimum cost edge but it should be connected to the vertices already present
(.) prim's algo always tries to maintain a tree.

MST COST = 99

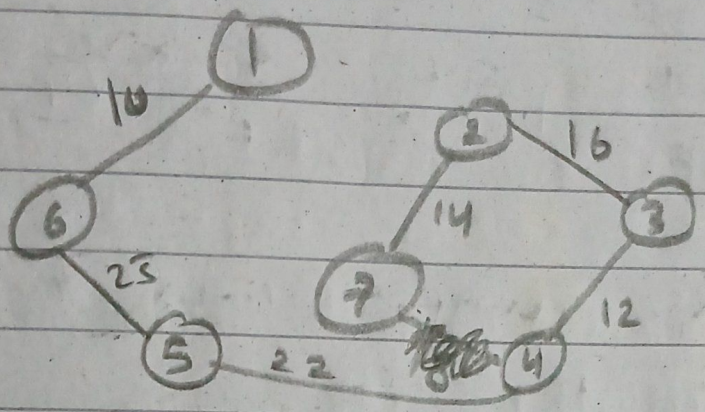




(.) We have no algo. to find the Minimum Cost Spanning tree for a non-connected graph.
(.) For a Spanning tree we need a connected graph.

KRUSKAL'S

1) Always select a minimum cost edge.



Cost = 98

2) But if that minimum cost edge that we are selecting form a cycle then we ignore it.

(•) Time COMPLEXITY = $O(|E||V|)$

$\approx O(e \cdot n) \approx O(n^2)$ →

↳ where e = no# of edges & n = no# of vertices

(•) $O(n^2)$ when both e & n are taken as n.

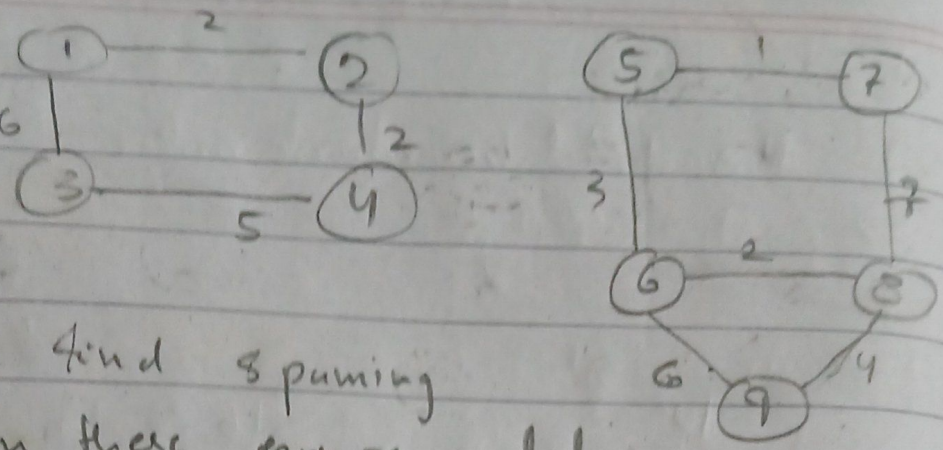
(•) But time complexity of ~~Prims~~ ~~Can~~ kruskal's can be improved by using Min Heap to store the edges in an order in which we always obtain the minimum edge.

↳ TC of deletion from MinHeap $O(\log n)$

↳ we need $|V|-1$ edges $\approx O(n)$

So TC $\approx O(n \log n)$

(o)



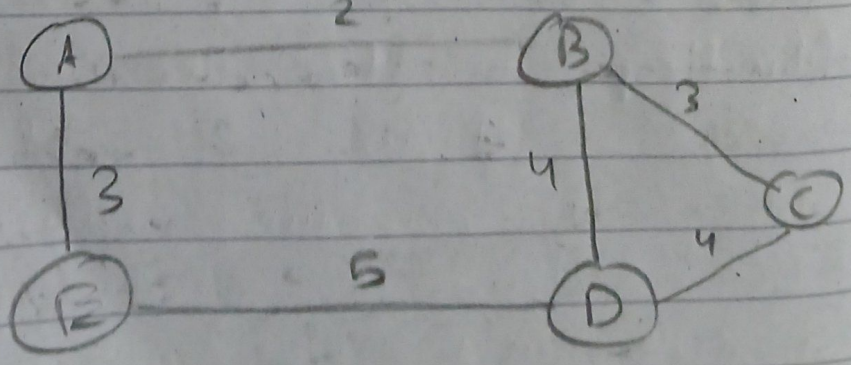
(-) Can we find spanning tree on these non-connected graphs?

(A) No, we can't find spanning trees on non-connected graphs

(.) But if we run KRUSHAL'S Alg. on this (to select $|V|-1$ edges)
(to select $|9|-1 = 8$ edges)

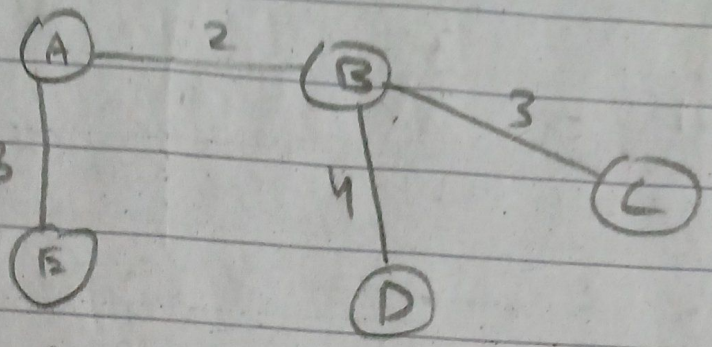
(o) It for sure won't find the spanning tree for the entire graph but it ~~will~~ may find spanning tree for each component but not for the entire graph (never).

Q)



$|V| = 5$ edges $= |V|-1 = 4$

A) KRUSKAL'S



Cost

↳ 2+3+3+4 = 12

↳ no# of edges to select in MCST = |V| - 1 where V = no# of vertices in graph

We only get 1 min / optimal solution

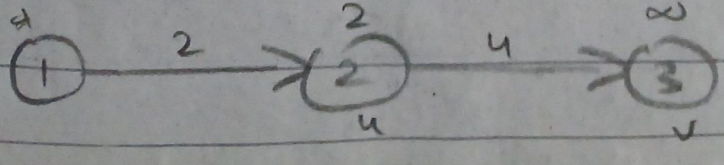
(•) In minimization problem we can only get 1 optimal cost but we may have more possible spanning trees.

DIJKSTRA'S ALGORITHM

(•) Algorithm for Single Source Shortest path problem

(•) finds the shortest path from one node to every other node.

(•) Whenever we select the shortest path we try & relax other vertices

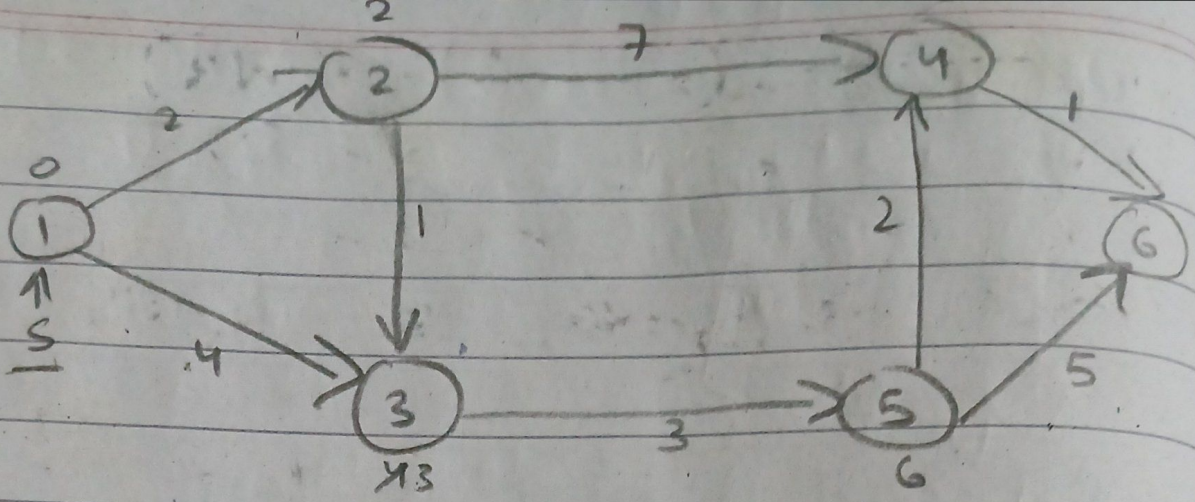


Relaxation

if(d[u] + (u,v) < d[v]

{ d[v] = d[u] + (u,v)

}



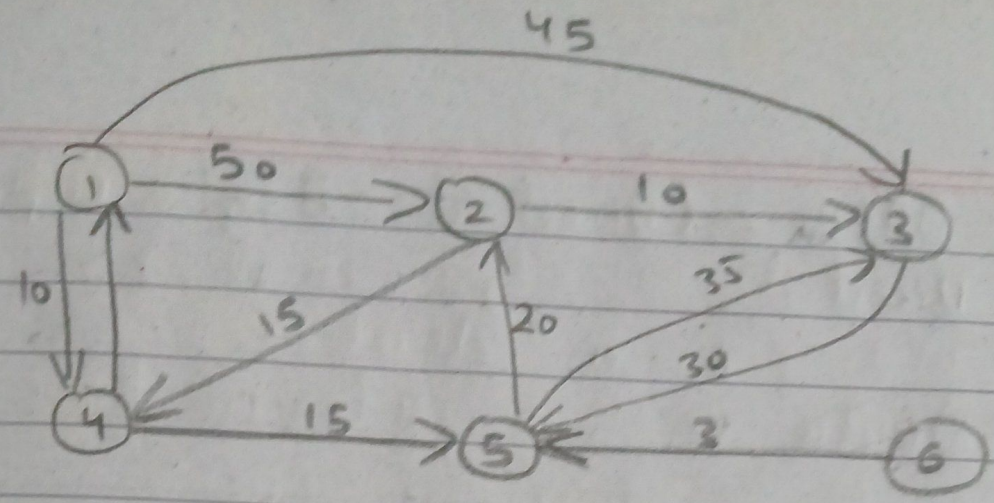
Step	N'	2	3	4	5	6
		d(2),p(2)	d(3),p(3)	d(4),p(4)	d(5),p(5)	d(6),p(6)
0	1	(2,1)	4,1	∞	∞	∞
1	12		(3,2)	9,2	∞	∞
2	123			9,2	(6,3)	∞
3	1235			(8,5)		11,5
4	12354					(9,4)
	123546					

(a) finds shortest path to all vertices
$n = |V|$

(b) whilst doing this the algo relaxes the vertices for us.

(c) Total no. of vertices relaxed?
we don't know, as this depends on the graph at most it will relax n vertices
$n = |V| \times |V|$
$= n \times n = n^2$ in case of complete graph

TC $O(n^2) \simeq O(|V^2|)$



Selected Vertex	2	3	4	5	6
	d(2), p(2)	d(3), p(3)	d(4), p(4)	d(5), p(5)	d(6), p(6)
1	50, 1	45, 1	10, 1	∞	∞
14	50, 1	45, 1		25, 4	∞
145	35, 5	45, 1			∞
1452		45, 1			∞
14523					∞
145236					∞

(•) path to vertex 6 is infinity.

DRAWBACK OF DIJKSTRA :-

(•) Doesn't account for / considered -ve edges. So it may or may not work for -ve edges.

(•) Greedy algo fails for -ve edges.

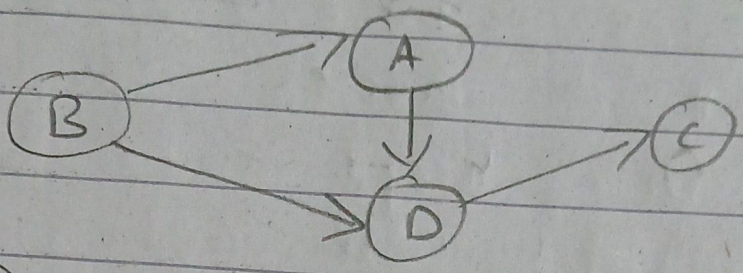
TOPOLOGICAL SORTING

(•) Can only be performed on DAG's graphs.

(.) DAGs (Directed Acyclic Graph)
(.) Directed → edges must have a direction
(.) Acyclic → no cycles in those graphs.

(.) We can't perform Topological ordering / sorting on undirected or graphs with cycles
(.) Linear ordering of all the vertices / nodes such that for every directed edge u→v, vertex u comes before v in the order
(.) There can be more than 1 correct answers
(.) Used to handle dependency problems / prereq for a chain course / OS software dependency problems / task scheduling problems
(.) Linear ordering of graph vertices such that for every directed edge uv from vertex u to vertex v, u comes before v in the ordering
(.) TIME COMPLEXITY ~~O(n)~~ O(V+E)

(c) It is like u is dependent upon v



Topological Order : BADC ✓

ABCD ✗

↳ edge is from B → A so

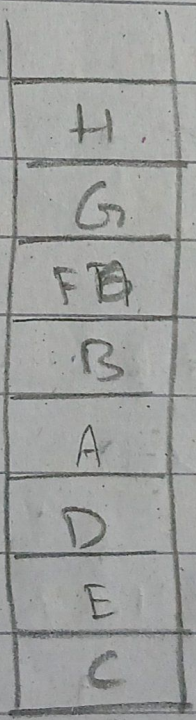
DFS Algo Based Approach

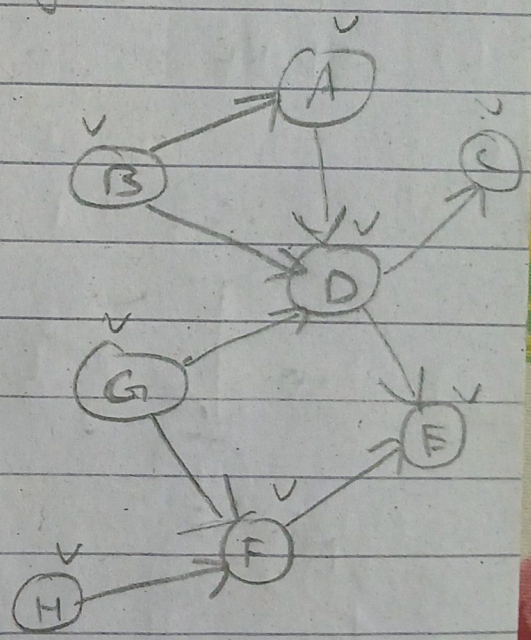
1) We can start from any state

dfs (B)

dfs (G)

dfs (H)





(a) Mark all visited nodes as visited.

(b) Keep on finding a path until you can't backtrack

(c) Once we reach a node at dead end / no path ahead, push the dead end

node into the stack

(•) Stack's size = size of the / total no# of nodes we have.

(•) loop of DFS keeps on repeating until all nodes are visited.

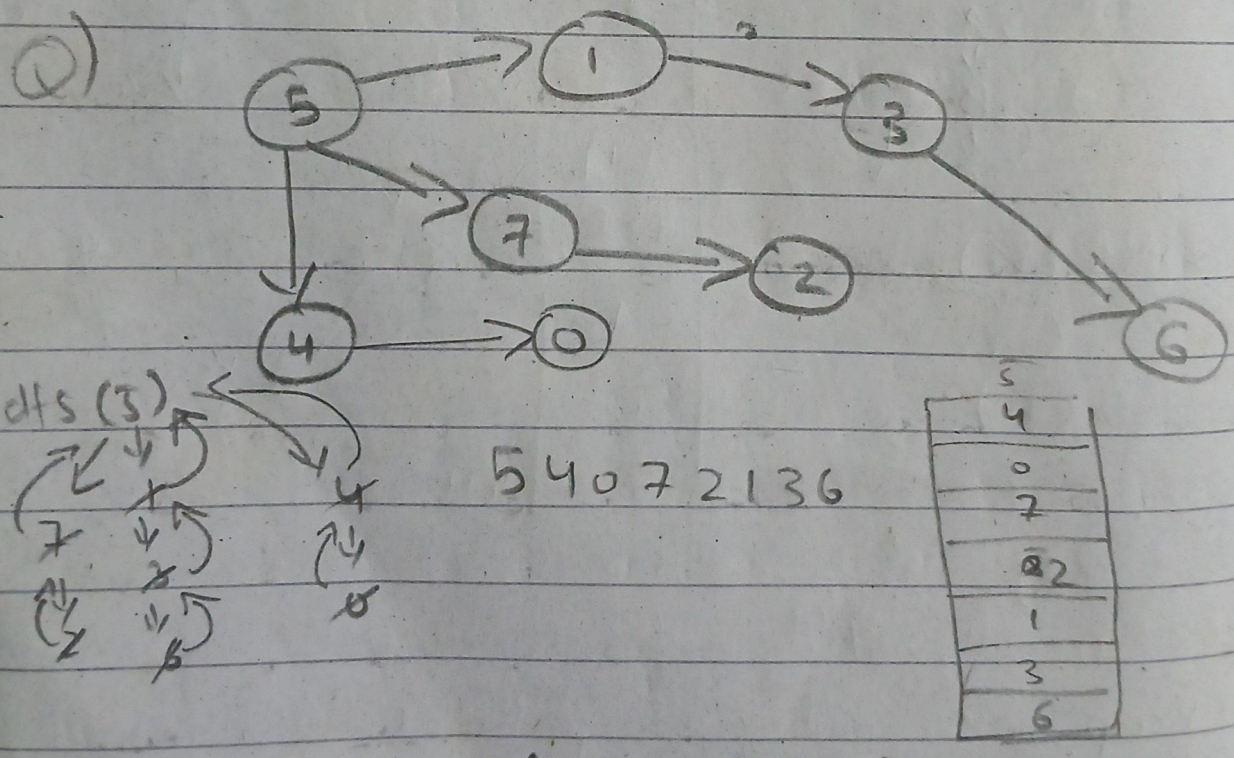
(•) pop the elements out from the stack HGFBADEC

(•) The ans is non unique but must be valid

(•) TIME COMPLEXITY O(V+E)

(•) SPACE " O(V)

Q)



(•) We can align all ~~edges to~~ nodes ~~the right~~ in a line & have all the edges pointing to the right

(6) Every trees (since they don't have cyles) have a topological ordering.